Alert Triage With Splunk

Use Splunk to triage alerts and investigate malicious activity efficiently.

Task 1 – Introduction

As a SOC analyst, it's important to be able to investigate different types of suspicious activity across a variety of assets in the environment. Knowing what to look for and which details matter most during an investigation is a key part of the role.

Learning Objectives

- Learn how to properly investigate alerts in a SOC environment.
- Understand how to investigate brute-force attacks on Linux systems.
- Discover the persistence mechanism on Windows systems.
- Analyse a web shell on a vulnerable web server.
- Learn how to investigate alerts for three given scenarios using Splunk.

Room Prerequisites

It is suggested to complete the following rooms first before proceeding:

- Introduction to SIEM
- Splunk: Basics
- Windows Logging Capabilities SIEM
- Log Analysis with SIEM
- Sysmon

Task 2 – Initial Access Alert

We will explore three different scenarios that you, as an analyst, may encounter during your shift. The first one will take place on a Linux host, and you will uncover what exactly happens there during our investigation.

Alert Scenario

> You've just started your first shift as a SOC analyst at an MSSP. only a few minutes have passed since an alert about a possible brute force attack appeared on the platform.
>
> **Alert Details:**
> - **Alert Name:** brute force activity detection
> - **Time:** 17/09/2025 9:00:21 am
> - **Target Host:** tryhackme-2404
> - **Source IP:** 10.10.242.248
>
> Your job is to investigate this activity and decide whether it should be considered suspicious.

The logs for this task are located in the Splunk index linux-alert. Use the following query: index=linux-alert

Investigating the Alert

Let's begin our investigation. First, we will focus on the alert data, in particular, two fields that may be of interest: **Target Host Time** and **Source IP**. Starting with the IP, as we can see, this is a local IP address. This suggests that, if it is indeed an attacker, they are already inside the organisation's network, possibly having successfully compromised the VPN or gained access through some other way. As for the host name, we don't have much information.

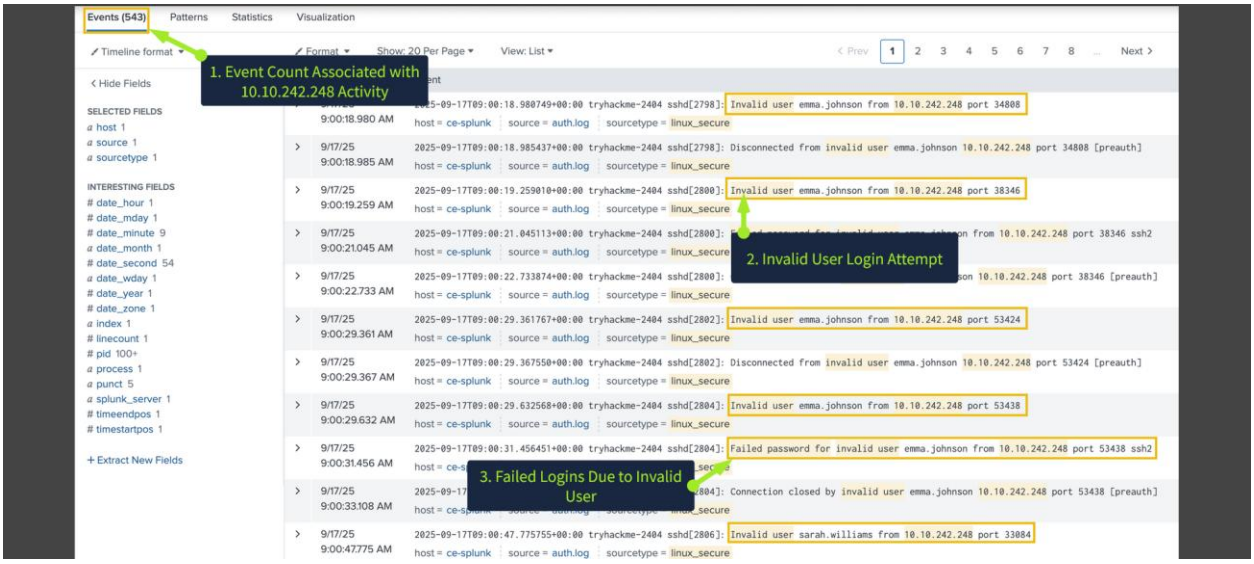
We lack an asset inventory table, and based on the hostname alone, it's hard to determine what system this is. Most likely, it's some type of organisational server. The activity time

appears normal, it’s 9 a.m., during regular working hours.
Therefore, let’s move into the SIEM and check whether brute force activity occurred here or if it's a false positive alert. Let's use this search in Splunk.

This query will search for both successful and failed login attempts, as well as events related to invalid users, which are commonly observed when an attacker is attempting to enumerate accounts before starting brute force attacks on a specific user.

```
index="linux-alert" sourcetype="linux_secure" 10.10.242.248
| search "Accepted password for" OR "Failed password for" OR "Invalid user"
| sort + _time
```

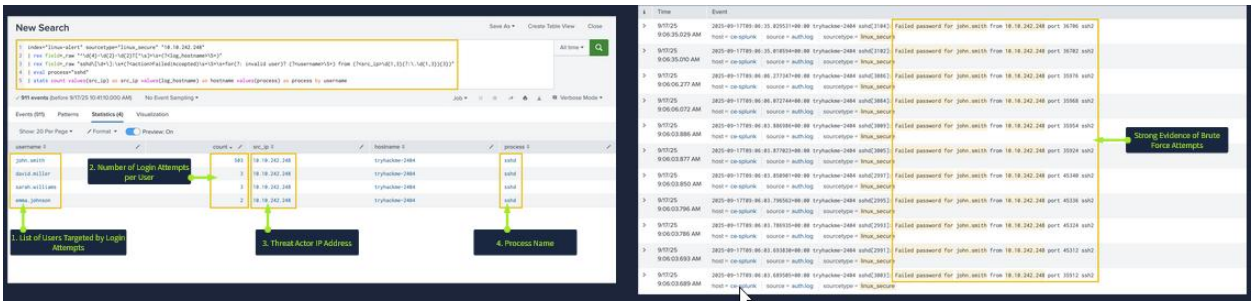
The first thing we notice is a rather large number of events associated with this IP. Secondly, what’s interesting is that there are several login-attempts for non-existent users, which is suspicious, wouldn’t you agree?



So far, we haven’t found any signs of brute force. Therefore, let’s run another search to see the number of login attempts made for each user.

```
index="linux-alert" sourcetype="linux_secure" 10.10.242.248
| rex field=_raw "^\d{4}-\d{2}-\d{2}T[^\s]+\s+(?<log_hostname>\S+)"
| rex field=_raw "sshd\[\d+\]:\s*(?<action>Failed|Accepted)\s+\S+\s+for(?: invalid user)? (?<username>\S+) from (?<src_ip>\d{1,3}(?:\.\d{1,3}){3})"
| eval process="sshd"
| stats count values(src_ip) as src_ip values(log_hostname) as hostname values(process) as process by username
```

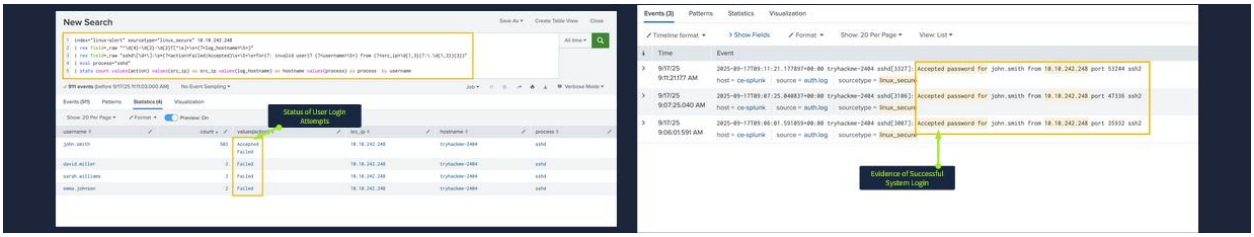
Now the results are more interesting. As we can see from the search output, the login attempts from this IP targeted four different users, but only **john.smith** shows a significantly high number of **503** attempts, which is a clear indicator of brute force activity.



We know that the brute force attempts targeted john.smith, but at this point, we don't yet know whether the attack was successful. Therefore, let's use the following search to determine whether the attacker actually gained access to the system.

```
index="linux-alert" sourcetype="linux_secure" 10.10.242.248
| rex field=_raw "^\d{4}-\d{2}-\d{2}T[^\s]+\s+(?<log_hostname>\S+)"
| rex field=_raw "sshd\[\d+\]:\s*(?<action>Failed|Accepted)\s+\S+\s+for(?: invalid user)? (?<username>\S+) from (?<src_ip>\d{1,3}(?:\.\d{1,3}){3})"
| eval process="sshd"
| stats count values(action) values(src_ip) as src_ip values(log_hostname) as hostname values(process) as process  by username
```

As a result, we detect a successful login only for the **john.smith** account, which gives us sufficient grounds to confirm that the brute force attack was successful and that the threat actor gained access to the host **tryhackme-2404**. We have clear indicators to classify this activity as a **True Positive**. In addition, we must immediately escalate this to the SOC L2 analyst for further investigation, with the possible involvement of the Incident Response team.



Next Investigation Steps

At this point, your investigation of this activity as a SOC Level 1 analyst comes to an end, since your responsibility is to identify malicious activity and escalate it to the Level 2 analyst. Therefore, let's now discuss what will happen next and what questions still remain for the SOC team regarding this case.

- Why did the attacker have a local IP address? Could it be that they are already inside our network? If so, for how long?
- How did the attacker obtain information about the users, specifically their usernames?
- What happened after the attacker gained access to the tryhackme-2404 host?

Following this, the Incident Response team should carry out a series of containment and remediation steps.
Not an easy shift you've got, huh!

Answer the questions below

How many failed login attempts were made on the user john.smith?

Query used:

```
index="linux-alert" sourcetype="linux_secure" 10.10.242.248 "john.smith" "Failed password"
| rex field=_raw "^\d{4}-\d{2}-\d{2}T[^\s]+\s+(?<log_hostname>\S+)"
| rex field=_raw "sshd\[\d+\]:\s*(?<action>Failed)\s+\S+\s+for(?: invalid user)? (?<username>\S+) from (?<src_ip>\d{1,3}(?:\.\d{1,3}){3})"
| eval process="sshd"
| search username="john.smith" action="Failed"
| stats count as failed_attempts values(src_ip) as src_ip values(log_hostname) as hostname values(process) as process by username
```

Figure 1 – Splunk query showing 500 failed SSH login attempts for john.smith

I filtered the Linux secure logs for failed SSH password attempts linked to john.smith. After extracting the username, source IP, hostname, and process from the raw events, I used stats count to summarize the results. As shown in Figure 1, there were 500 failed SSH login attempts from 10.10.242.248.

Answer: 500

What was the duration of the brute force attack in minutes?

Query used:

index="linux-alert" sourcetype="linux_secure" 10.10.242.248 "john.smith" "Failed password"

| rex field=_raw "sshd\[\d+\]:\s(?<action>Failed)\s+\S+\s+for(?: invalid user)? (?<username>\S+) from (?<src_ip>\d{1,3}(?:\.\d{1,3}){3})"*

| search username="john.smith" action="Failed"

| stats earliest(_time) as first_attempt latest(_time) as last_attempt count as failed_attempts by username src_ip

| eval duration_seconds=last_attempt-first_attempt

| eval duration_minutes=round(duration_seconds/60,2)

| convert ctime(first_attempt) ctime(last_attempt)

| table username src_ip failed_attempts first_attempt last_attempt duration_minutes

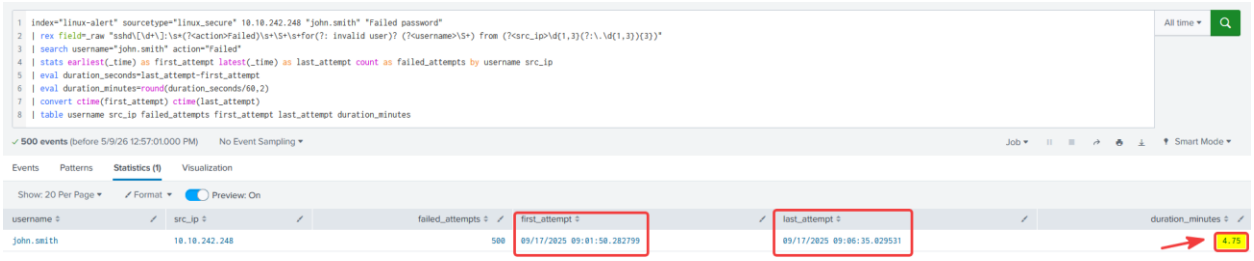


Figure 2 – Calculating the brute-force attack duration in Splunk

To calculate the duration of the brute-force attack, I reused the failed SSH login search for john.smith. The query used earliest(_time) to find the first failed login attempt and latest(_time) to find the last failed attempt. I then subtracted the two timestamps and converted the result from seconds into minutes. As shown in Figure 2, the attack lasted 4.75 minutes, which rounds to 5 minutes.

Answer: 5

What username was the attacker able to privilege escalate to?

Query used:

```
index="linux-alert" sourcetype="linux_secure" ("sudo" OR "su" OR "session opened")
| rex field=_raw "sudo:\s+(?<src_user>\S+)\s+:.*USER=(?<target_user>\S+)"
| rex field=_raw "su:\s+.*session opened for user (?<su_target>\S+)"
| eval escalated_user=coalesce(target_user, su_target)
| search src_user="john.smith" OR _raw="*john.smith*"
| table _time src_user escalated_user host _raw
| sort _time
```

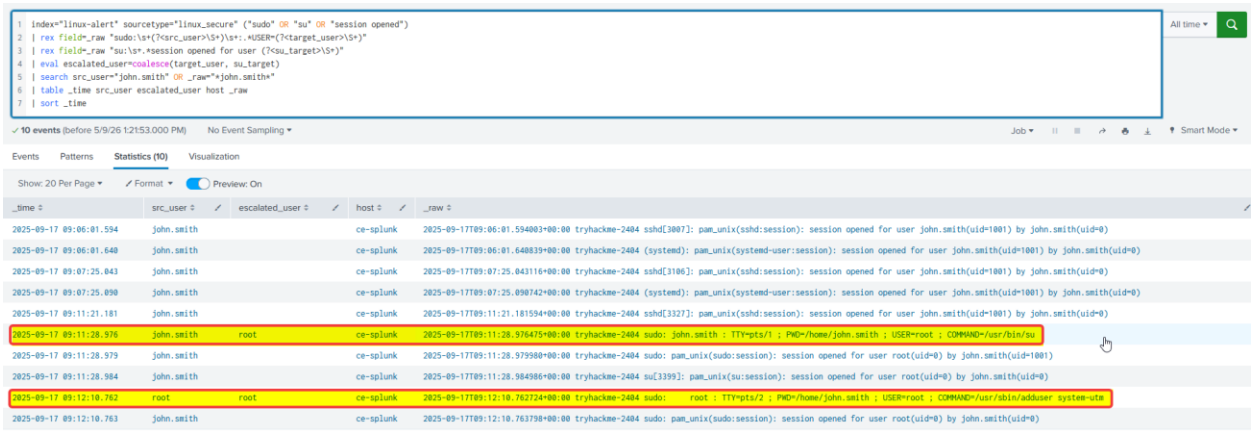


Figure 3 – Splunk results showing privilege escalation from john.smith to root

I searched the Linux secure logs for privilege escalation activity using sudo, su, and session-related events. The results showed john.smith running /usr/bin/su with USER=root, followed by a session being opened for root. This confirms that the attacker successfully escalated privileges to the root account.

Answer: root

What is the name of the user account created by the attacker for persistence?

Query used:

index="linux-alert" sourcetype="linux_secure" ("adduser" OR "useradd")

| table _time host _raw

| sort _time

Figure 4 – Splunk evidence showing the attacker creating the system-utm user account

The logs showed the attacker running adduser system-utm after gaining root privileges. A following useradd event confirmed that a new user account named system-utm was created, indicating persistence activity.Answer: system-utm

Task 3 – Persistence Alert

The second scenario will focus on activity in a Windows environment, which SOC analysts come across on a regular basis. That's why knowing how to carry out the analysis is really important for you.

Alert Scenario

You are working as a Level 1 Analyst on shift at an MSSP. An alert has come through indicating that a suspicious scheduled task was created on a host.

Alert Details:

- **Alert Name:** Potential Task Scheduler Identified
- **Time:** 30/08/2025 10:06:07 AM
- **Host:** WIN-H015
- **User:** oliver.thompson
- **Task Name:** AssessmentTaskOne

Your job is to investigate this activity and decide whether it should be considered suspicious.

The logs for this task are located in the Splunk index win-alert. Use the following query: index=win-alert

Investigating the Alert

Let's look at how to approach this analysis. First, pause - don't jump straight into the SIEM. Begin with the alert details.

Focus on **Host** and **User**. Work out what kind of host it is: workstation or server. This context is key before continuing. Many organisations have an Asset Inventory (sometimes in Confluence or Notion) for this, but if not, naming conventions can help.

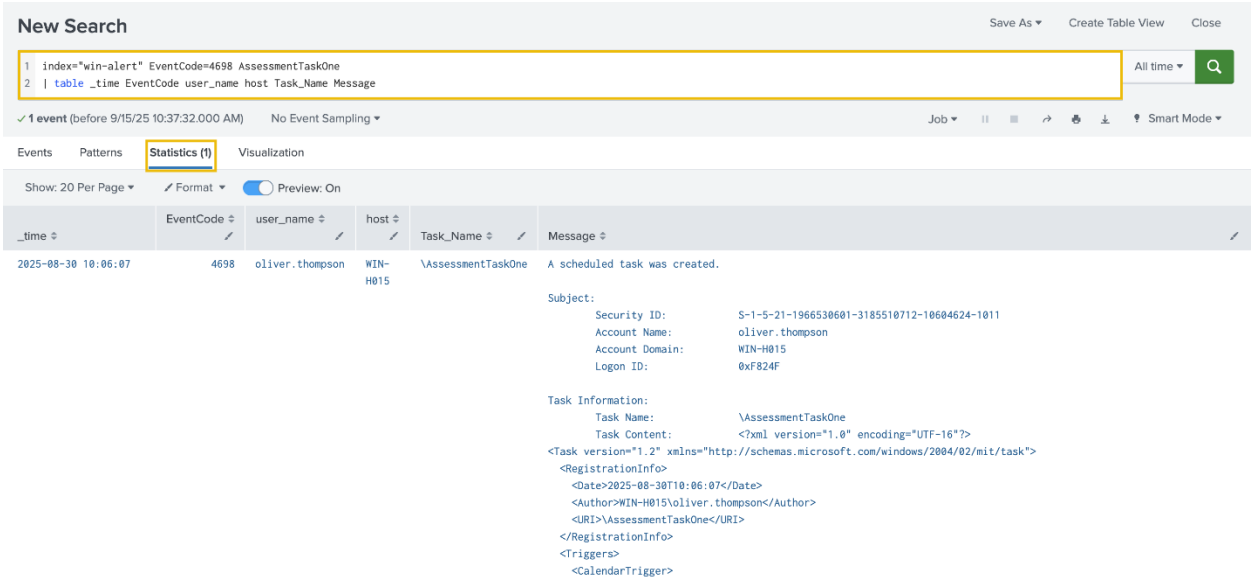
Servers often use prefixes like **SRV**, **WEB**, **MSQL**, while workstations are labelled **WIN** or **HOST**. In our case, **WIN-H015** suggests it's a workstation.

Next, check the **User**. Use the identity table to see their role. This helps decide if the activity fits their job. For example, it's odd if HR is creating tasks, but normal for IT staff. You can also check the user's location and working hours to see if the timing makes sense. In our case, **Oliver Thompson** is a System Engineer.
Once we've gathered this information, we will have an initial picture of where the activity took place and who carried it out. Now, let's move into the SIEM for a deeper analysis.

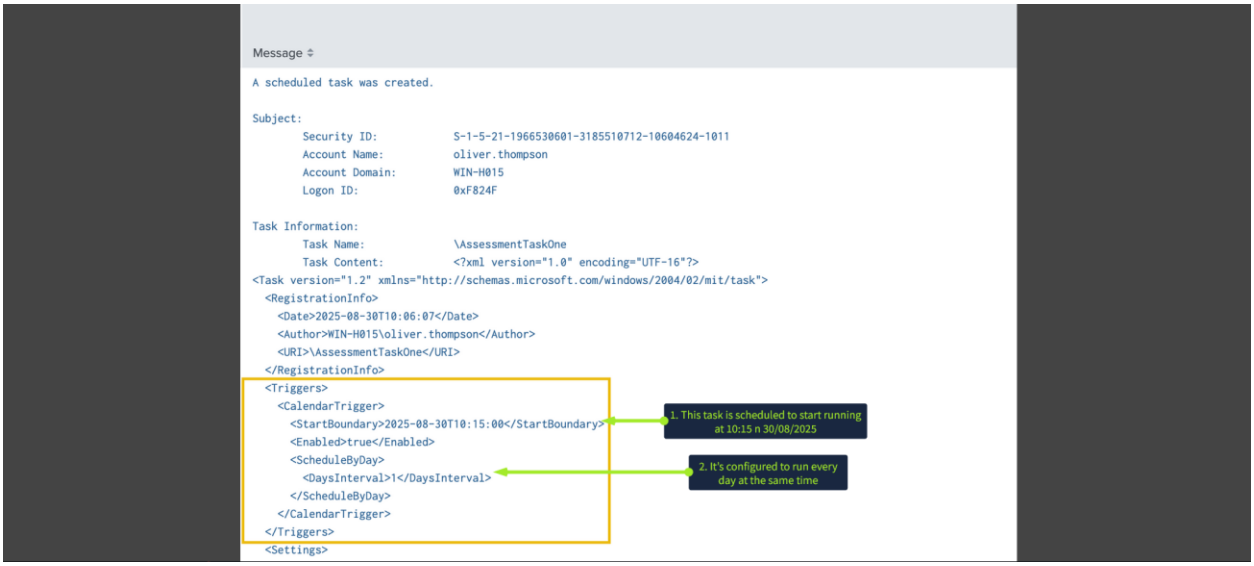
To start, we'll query the task name **AssessmentTaskOne** along with event ID **4698**, which indicates that a scheduled task was created. Since we also know the exact time of the activity, we can filter by timestamp to speed up the search. **Note:** For now, we won't filter by host. This way, we can check whether the activity is isolated to a single machine or present across multiple hosts.

```
index="win-alert" EventCode=4698 AssessmentTaskOne
| table _time EventCode user_name host Task_Name Message
```

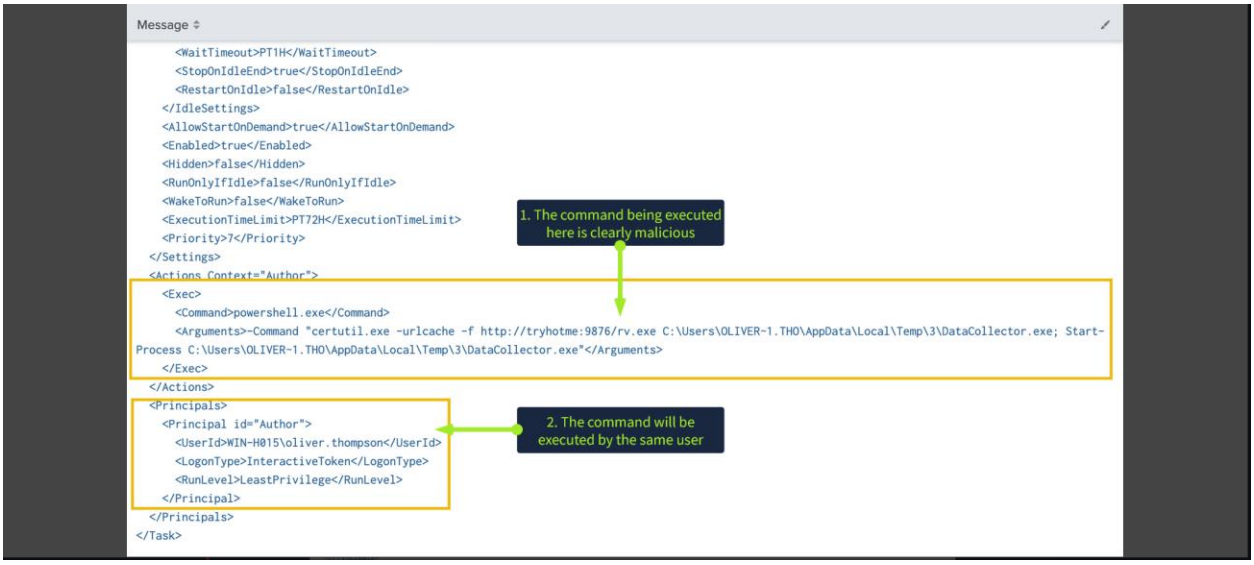
From our search, we can see that there's only a single event related to this activity.



Now, let's look at the **Message** field to understand what this task actually does. Let's go step by step, starting with the **Triggers** section.



We can see that the task runs every day at the same time on a user workstation, which is quite unusual. Let's continue by analysing the **Message** field, focusing on the **Exec** and **Principals** sections, to see what task is being executed and under which user account.



At this point, we can already see the first signs of malicious activity. This task will use **certutil** to download **rv.exe** from the **tryhotme** domain into the Temp folder under the name **DataCollector.exe**. It will then launch this file using a **Start-Process** PowerShell command. All of this activity will be executed under the user **oliver.thompson**. This is a clear example of persistence.

What else can you find out? Try looking up the domain in Threat Intelligence platforms, it might be linked to a known attacker infrastructure. We classified this activity as a **True Positive**. Also, this activity should be escalated to an L2 analyst for deeper investigation.

Next Investigation Steps

Of course, there are still open questions, such as:

- How was this scheduled task created?
- How did the attacker gain access to the WIN-H015 host?
- How was the oliver.thompson account compromised?

Normally, these questions would be addressed by an SOC L2 analyst. However, since this is a training exercise, you'll have the chance to answer them yourself during the practical part of this task.

Answer the questions below

What is the ProcessId of the process that created this malicious task?

Query used:

index="win-alert" EventCode=4698 AssessmentTaskOne

| table _time EventCode user_name host Task_Name ClientProcessId Message

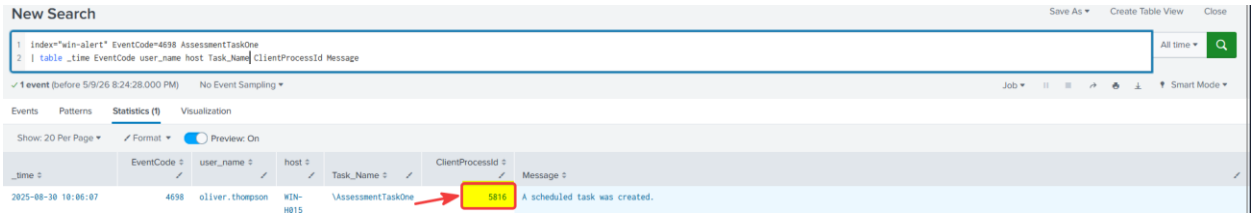


Figure 5 – Splunk result showing the ClientProcessId for the AssessmentTaskOne scheduled task

I searched Windows Security Event ID 4698, which records scheduled task creation, and filtered for the task AssessmentTaskOne. The result showed that the task was created by oliver.thompson, with a ClientProcessId of 5816.

Answer: 5816

What is the name of the parent process for the process that created this malicious task?

Query used:

index="win-alert" EventCode=1 ProcessId=5816

| table _time host user_name Image ProcessId ParentImage ParentProcessId CommandLine ParentCommandLine

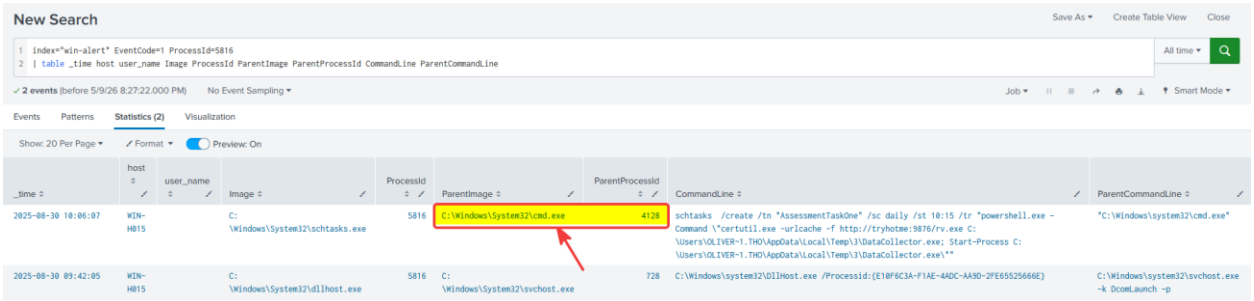


Figure 6 – Splunk result showing cmd.exe as the parent process of schtasks.exe

After identifying 5816 as the ClientProcessId that created the scheduled task, I searched for the matching process creation event. The result showed schtasks.exe running with ProcessId 5816, and its ParentImage was C:\Windows\System32\cmd.exe, confirming that the parent process was cmd.exe.

Answer: cmd.exe

Which local group did the attacker enumerate during discovery?

Query used:

index="win-alert" EventCode=1 "oliver.thompson"

*| search CommandLine="*localgroup**"*

| table _time host user_name Image CommandLine ParentImage

| sort _time

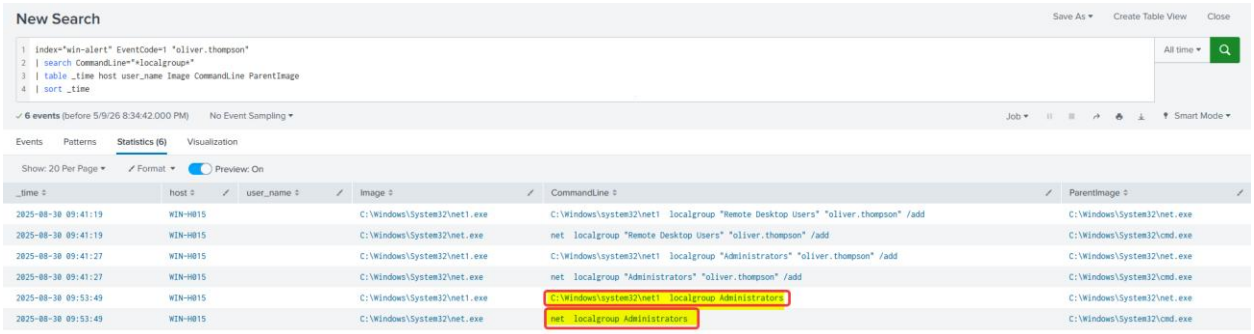


Figure 7 – Splunk results showing enumeration of the local Administrators group

I searched Windows process creation events for localgroup commands linked to oliver.thompson. The results showed the attacker running net localgroup Administrators, which enumerates the members of the local Administrators group. This confirmed that the local group enumerated during discovery was Administrators.

Answer: Administrators

What is the name of the workstation from which the Threat Actor logged into this host?

Query used:

index="win-alert" EventCode=4624 "oliver.thompson"

| table _time host Account_Name Logon_Type Workstation_Name Source_Network_Address IpAddress

| sort _time

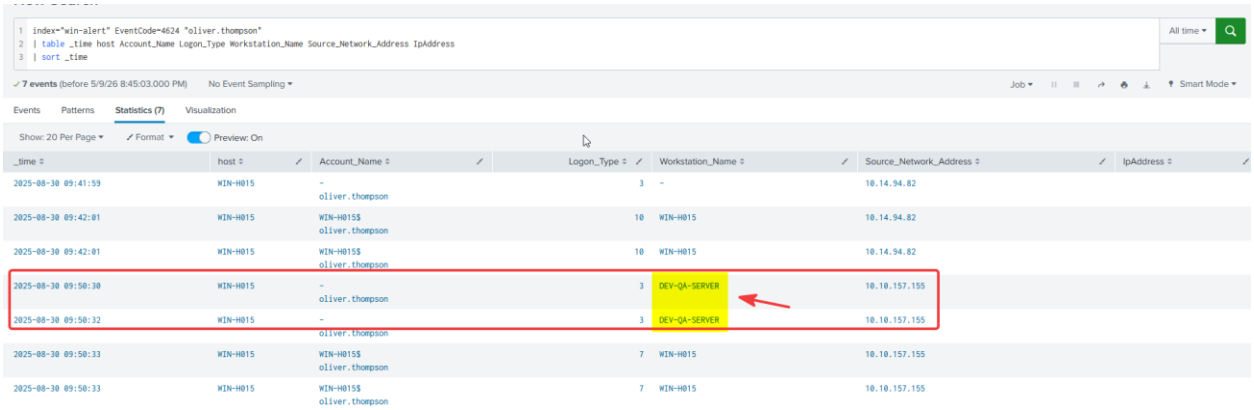


Figure 8 – Splunk results showing DEV-QA-SERVER as the workstation used for logon

I searched Windows Security Event ID 4624 for successful logons involving oliver.thompson. I removed the full Message field to keep the output readable and focused on the account name, logon type, workstation name, and source IP. The results showed the threat actor logging in from workstation DEV-QA-SERVER.

Answer: DEV-QA-SERVER

Task 4 – Web Shell Alert

The last scenario will focus on activity in a vulnerable web server. We'll be investigating possible web shell exploitation.

Alert Scenario

Your shift as an L1 analyst continues, and you've now received the next alert that needs to be investigated. This time, the activity is related to the web.

Alert Details:

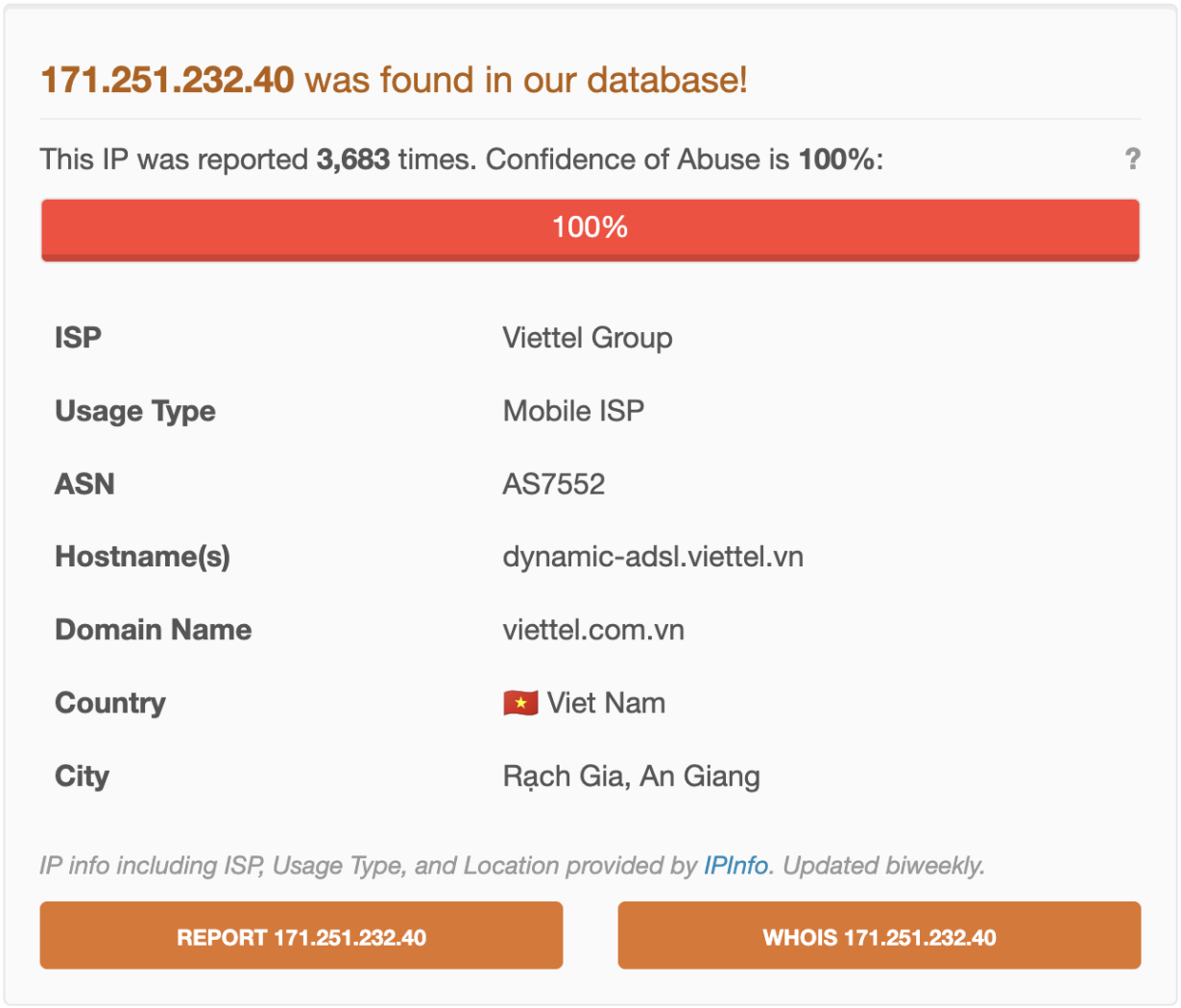
- **Alert Name:** Potential Web Shell Upload Detected
- **Time:** 14/09/2025 09:31:51 AM
- **Resource:** ://web.trywinme.
- **Suspicious IP:** 171.251.232.40

Your job is to investigate this activity and decide whether it should be considered suspicious.

The logs for this task are located in the Splunk index win-web. Use the following query: index=web-alert

Investigating the Alert

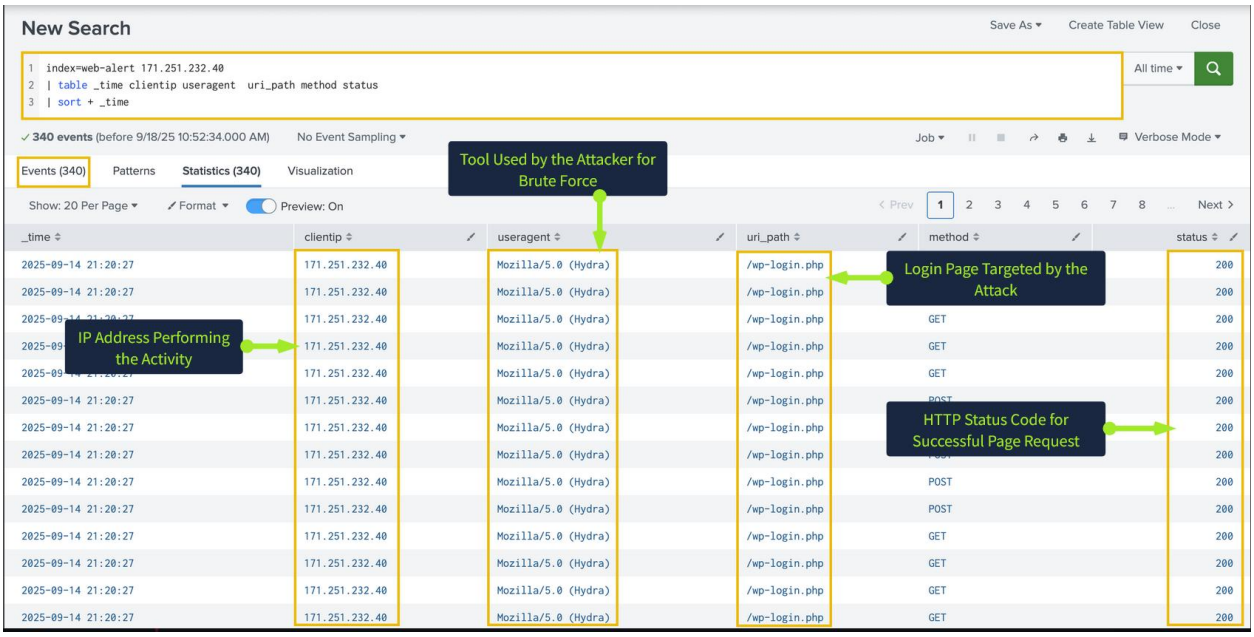
As with the previous cases, we begin by examining the details contained in the alert. The first thing of interest is the resource where the activity occurred, in this case, http://web.trywinme.thm, which is the organisation's website hosted on the web server. The next point of interest here is the Suspicious IP address. We can check it across various Threat Intelligence platforms to gather more information. Let's use AbuseIPDB (opens in new tab) for this. As we can see, this IP address has been flagged as malicious more than 3000 times. Let's make a note of this.



Now, let's move into the SIEM to examine the specific activity carried out by this IP address.

```
index=web-alert 171.251.232.40
| table _time clientip useragent uri_path method status
| sort + _time
```

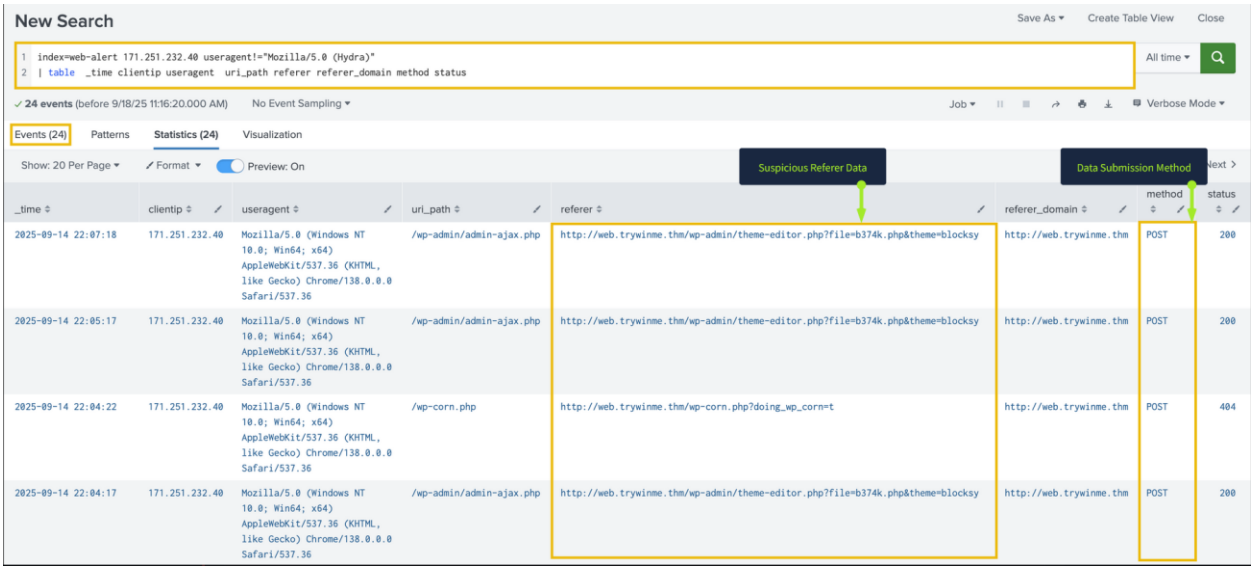
Right away, we can detect a large number of requests associated with this IP. What's particularly interesting is that the User-Agent is set to Hydra, a popular tool commonly used by attackers to perform **brute force** attempts, in our case, against the **wp-login.php** page. This is already a clear indicator of malicious activity, specifically attempts to gain unauthorized access. However, our alert is about a web shell, so let's take a closer look at what else has occurred.



We excluded Hydra as the User-Agent from our search and reviewed the results.

index=web-alert 171.251.232.40 useragent!="Mozilla/5.0 ()"
| table _time clientip useragent uri_path referer referer_domain method status

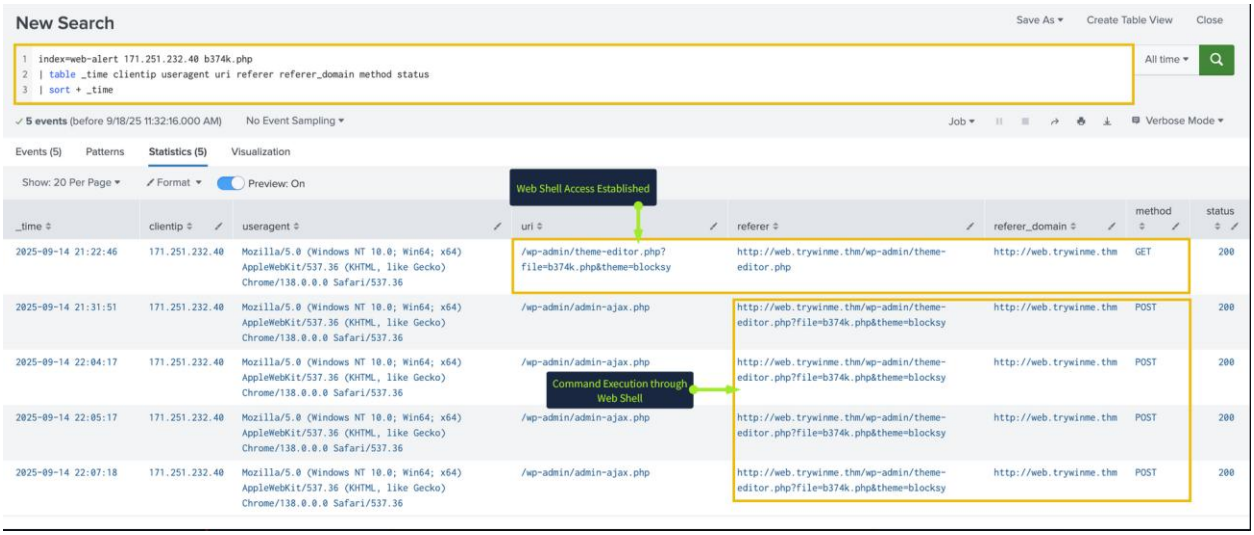
When reviewing the search results, one detail immediately stands out. A **POST** request was observed for **admin-ajax.php** with a referer pointing to **theme-editor.php?file=b374k.php**. This is unusual because the theme editor should not reference arbitrary .php files. The presence of **file=b374k.php** strongly suggests that the attacker may have uploaded or is interacting with a web shell.



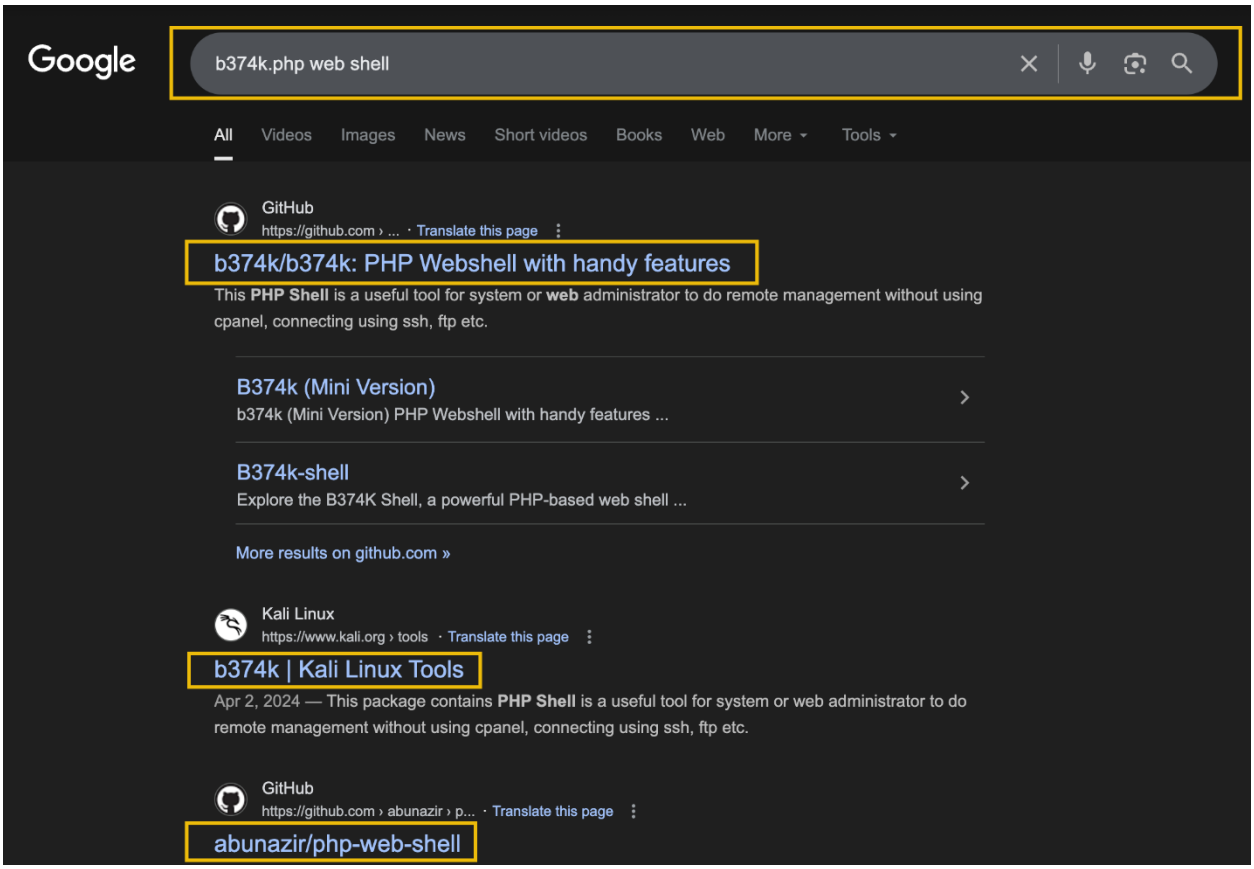
Let’s now take a closer look at the logs related to b374k.php.

```
index=web-alert 171.251.232.40 b374k.php
| table _time clientip useragent uri_path referer referer_domain method status
| sort + _time
```

We detected that the threat actor successfully gained access to a possible web shell file, **b374k.php**. After that, they began executing activity through it, specifically, we observed four successful **POST** requests. Unfortunately, the logs do not show how the attacker initially uploaded the web shell to the server.



Sometimes attackers use popular web shells without even changing their filenames. Let’s do some research on the web shell we’ve identified. Just Google: "b374k.php web shell".



This finding provides additional confirmation that the identified file is a web shell. In summary, we identified malicious activity originating from a Vietnamese IP address **171.251.232.40** targeting the web server. The suspicious activity began with a brute force attack using Hydra against **wp-login.php**, followed by clear evidence of web shell activity involving **b374k.php**. We can confidently classify this activity as a **True Positive** and escalate it to a SOC Level 2 analyst, as this case requires deeper investigation and will undoubtedly involve the Incident Response team.

Next Investigation Steps

These questions should be reviewed by the SOC Level 2 analyst:

- Was the brute force attack using Hydra successful?
- How did the attacker upload the web shell to the server, given that SOC L1 did not identify any traces of the upload?

- What specific actions did the attacker perform on the server using commands through the web shell?

After this, containment and remediation actions should take place.

That was quite an intense shift. I hope you've learned how to properly conduct an investigation and what you can identify in the logs.

Answer the questions below

What time did the brute-force activity using Hydra begin?
Answer Format Example: 2025-01-15 12:30:45

Query used:

index="web-alert" 171.251.232.40 "Hydra" method="POST"

| stats earliest(_time) as first_seen latest(_time) as last_seen count as total_attempts by clientip useragent method uri_path

| convert ctime(first_seen) ctime(last_seen)

| table clientip useragent method uri_path total_attempts first_seen last_seen

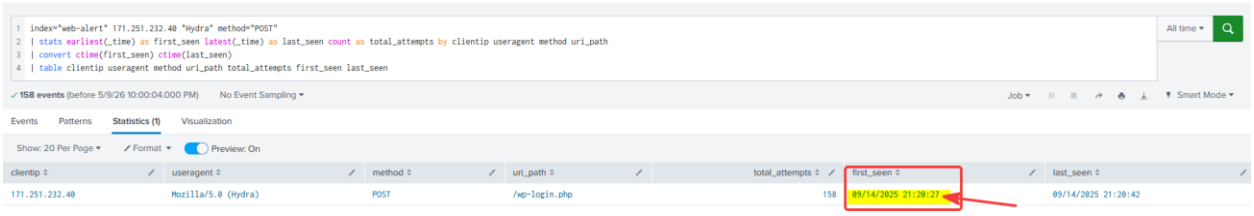


Figure 9 – Splunk results showing the start time of the Hydra brute-force activity

I filtered the web logs for POST requests from 171.251.232.40 with the Hydra user agent. Since the requests targeted /wp-login.php, the POST method helped focus on actual login submission attempts. I used earliest(_time) to identify when the brute-force activity began. The first Hydra POST request was recorded at 09/14/2025 21:20:27.

Answer: 2025-09-14 21:20:27

Which user agent did the attacker use when interacting with the web shell?

Query used:

index="web-alert" 171.251.232.40 b374k.php

| stats count as requests values(useragent) as useragent by clientip

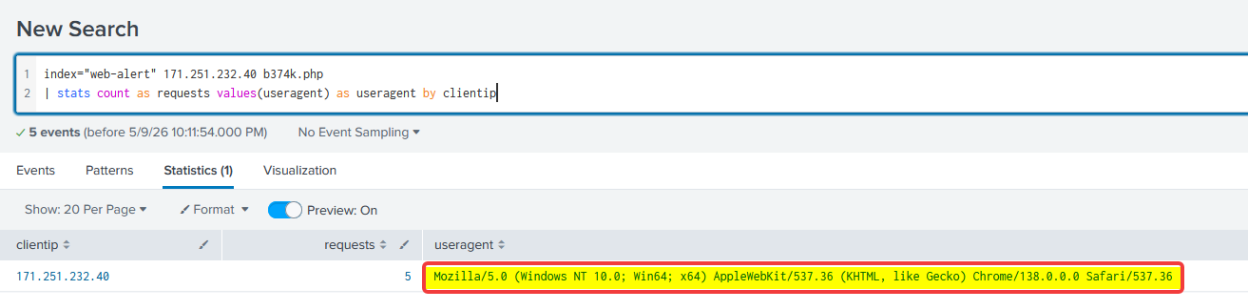


Figure 10 – Splunk results showing the user agent used to access the b374k.php web shell

I searched the web logs for requests to b374k.php, which identified interaction with the web shell. The results showed the attacker using a Chrome-based user agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36.

Answer: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36

What was the number of requests made by the attacker to the server via the web shell?

Query used:

index="web-alert" b374k.php

| stats count as web_shell_requests by clientip uri_path

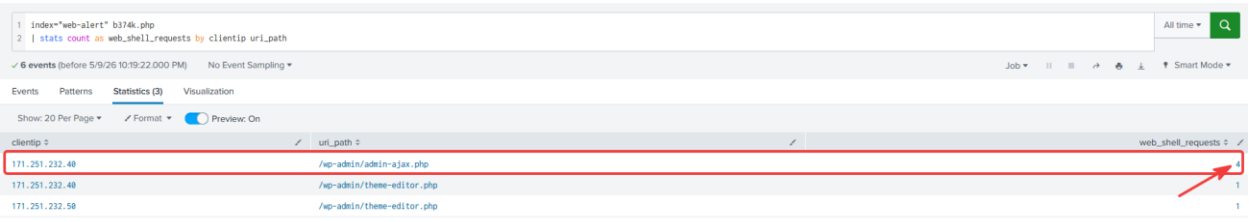


Figure 11 – Splunk results showing 4 web shell requests from the attacker IP

The attacker IP 171.251.232.40 made 4 requests through /wp-admin/admin-ajax.php, confirming the number of web shell requests.

Answer: 4

Conclusion

I've now gained practical experience in investigating different types of alerts analysts can encounter in the real world.

- Detecting suspicious activity on Windows and Linux systems.
- Analysing web shell activity and identifying its traces.